Лабораторная работа №1. Регрессионный анализ

Студент: Васютинский Вадим Александрович **Вариант:** 8 (`lab1-08.csv`)

Цель: научиться применять модели линейной регрессии библиотеки scikit-learn к набору эмпирических данных.

Результат: эмпирическая формула зависимости $y(x)$.

In [1]:
```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, FunctionTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_s

RANDOM_STATE = 42
DATA_PATH = Path('..') / '..' / 'tasks' / 'lab1' / 'dataset' / 'lab1-08.c
```

1. Загрузка данных

In [2]:
```
data = pd.read_csv(DATA_PATH, header=None, names=['x', 'y'])
data.head()
```

Out[2]:

	x	y
0	7.393939	-4.832168
1	-2.595960	21.935301
2	-1.030303	8.319461
3	7.454545	-3.898417
4	-2.292929	16.866421

In [3]:
```
data.info()
data.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   x       200 non-null    float64
 1   y       200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

Out[3]:

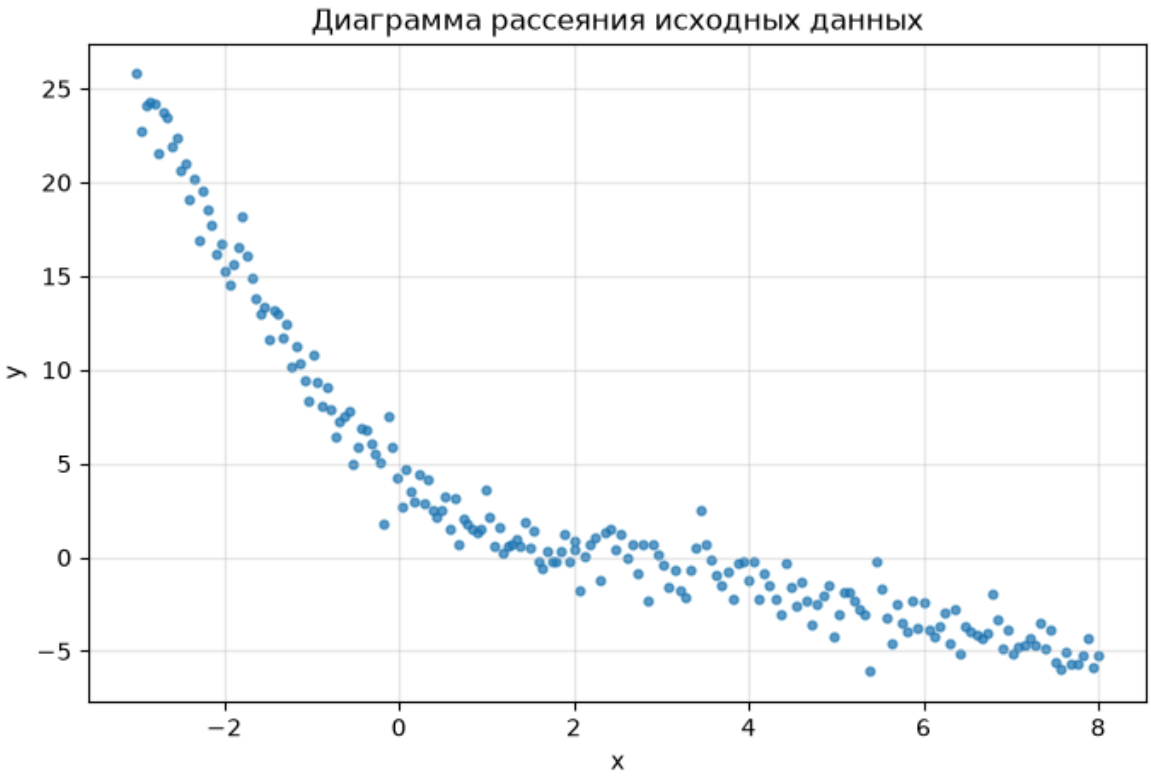
	x	y
count	200.000000	200.000000
mean	2.250000	3.245133
std	3.194769	7.961822
min	-3.000000	-6.088947
25%	-0.487374	-2.308487
50%	2.000000	0.580878
75%	4.984848	6.787182
max	8.000000	25.796033

Файл содержит 200 наблюдений и два числовых столбца без пропусков: признак $x \in [-3, 8]$ и целевую переменную y.

2. Визуализация исходных данных

In [4]:

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(data['x'], data['y'], s=12, alpha=0.7)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Диаграмма рассеяния исходных данных')
ax.grid(alpha=0.3)
plt.show()
```



Зависимость явно нелинейная: при $x < 1$ значения y резко убывают (по форме похоже на экспоненту e^{-kx}), а при $x > 1$ убывание становится медленным и почти линейным, без выхода на горизонтальную асимптоту. Поэтому кроме обычной прямой рассмотрим две модели, линейные по параметрам:

1. полиномиальную регрессию $y = \sum_{i=0}^{n} w_i x^i$;
2. регрессию на признаках e^{-kx} и x: $y = ae^{-kx} + bx + c$.

Обе модели обучаются с помощью `LinearRegression`, нелинейность вносится через преобразование признаков.

3. Разбиение на обучающую и тестовую выборки

In [5]:
```
X = data[['x']]
y = data['y']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)
print(f'Обучающая выборка: {len(X_train)}, тестовая: {len(X_test)}')
```

```
Обучающая выборка: 160, тестовая: 40
```

Тестовая выборка (20 %) используется только для итоговой оценки. Гиперпараметры (степень полинома, коэффициент k) подбираются кросс-валидацией на обучающей выборке.

4–5. Выбор и обучение моделей

In [6]:
```
cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)


def evaluate(model, name):
    """Считает метрики модели на обучающей и тестовой выборках."""
    rows = []
    for split, X_part, y_part in [('train', X_train, y_train), ('test', X
        pred = model.predict(X_part)
        rows.append({
            'model': name,
            'split': split,
            'MSE': mean_squared_error(y_part, pred),
            'MAE': mean_absolute_error(y_part, pred),
            'R2': r2_score(y_part, pred),
        })
    return rows
```

5.1. Базовая модель: простая линейная регрессия $y = w_1 x + w_0$

In [7]:
```
linear = LinearRegression().fit(X_train, y_train)
print(f'y = {linear.coef_[0]:.4f} * x + {linear.intercept_:.4f}')
```

```
y = -2.1636 * x + 8.0350
```

5.2. Полиномиальная регрессия: подбор степени

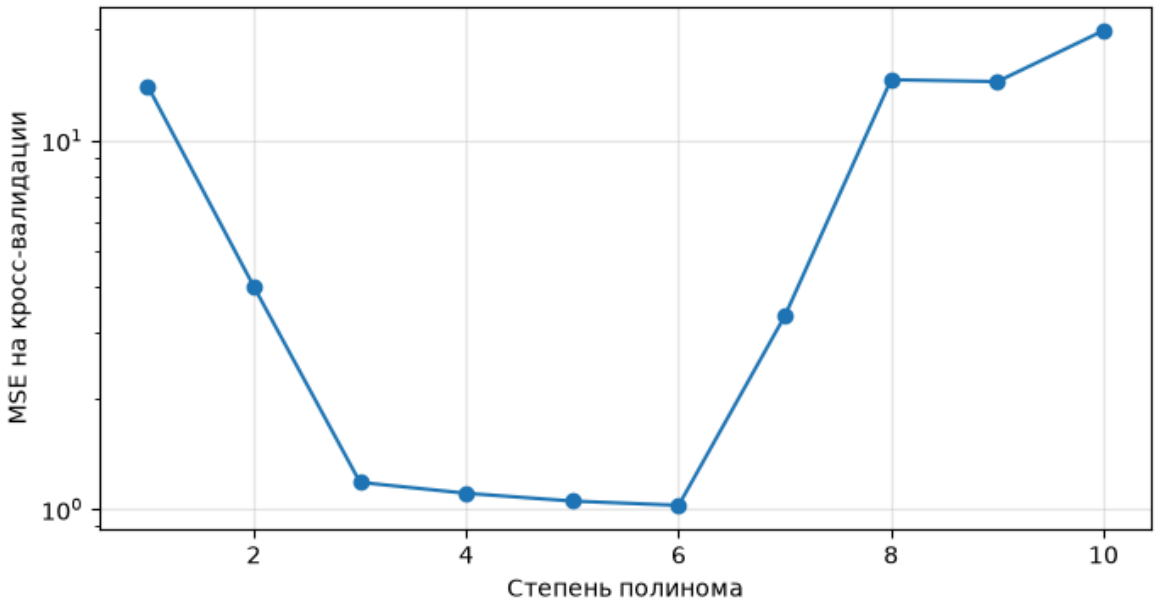
In [8]:
```
poly_pipeline = Pipeline([
    ('poly', PolynomialFeatures(include_bias=False)),
    ('linreg', LinearRegression()),
])
poly_search = GridSearchCV(
    poly_pipeline,
    {'poly__degree': range(1, 11)},
    scoring='neg_mean_squared_error',
    cv=cv,
)
poly_search.fit(X_train, y_train)

poly_cv = pd.DataFrame({
    'degree': poly_search.cv_results_['param_poly__degree'].astype(int),
    'cv_MSE': -poly_search.cv_results_['mean_test_score'],
})
poly_cv
```

Out[8]:

	degree	cv_MSE
0	1	14.049732
1	2	4.007759
2	3	1.180182
3	4	1.102258
4	5	1.049963
5	6	1.022504
6	7	3.339814
7	8	14.594797
8	9	14.422262
9	10	19.835017

In [9]:
```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(poly_cv['degree'], poly_cv['cv_MSE'], marker='o')
ax.set_xlabel('Степень полинома')
ax.set_ylabel('MSE на кросс-валидации')
ax.set_yscale('log')
ax.grid(alpha=0.3)
plt.show()
```



Ошибка резко падает до степени 3, на степенях 4–6 уменьшается незначительно, а начиная со степени 7 снова растёт: полином высокой степени подстраивается под шум и сильно колеблется на краях отрезка. Возьмём наименьшую степень, у которой ошибка на кросс-валидации отличается от минимальной не более чем на 10 %.

In [10]:
```
best_cv_mse = poly_cv['cv_MSE'].min()
best_degree = int(poly_cv.loc[poly_cv['cv_MSE'] <= 1.1 * best_cv_mse, 'de
print('Выбранная степень полинома:', best_degree)

poly_model = Pipeline([
```

```
    ('poly', PolynomialFeatures(degree=best_degree, include_bias=False)),
    ('linreg', LinearRegression()),
]).fit(X_train, y_train)
```

```
Выбранная степень полинома: 4
```

5.3. Регрессия на признаках e^{-kx} и x: подбор k

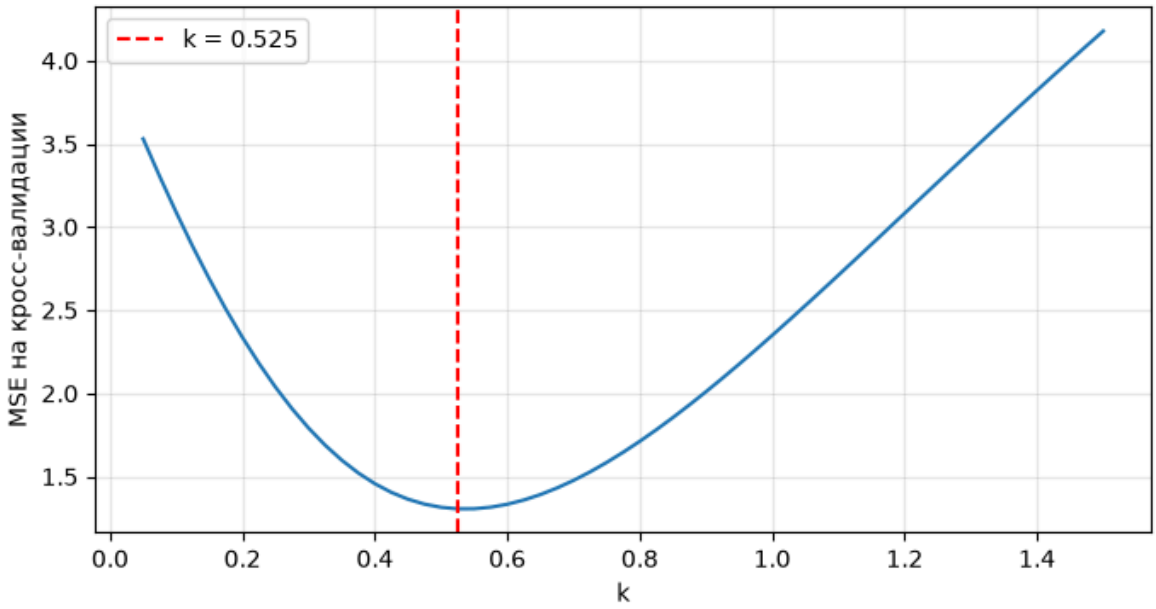
In [11]:
```
def exp_linear_features(X, k):
    """Переводит x в признаки [exp(-k*x), x]."""
    x = np.asarray(X, dtype=float).ravel()
    return np.column_stack([np.exp(-k * x), x])


exp_pipeline = Pipeline([
    ('features', FunctionTransformer(exp_linear_features, kw_args={'k': 1
    ('linreg', LinearRegression()),
])
k_grid = np.round(np.linspace(0.05, 1.5, 59), 3)
exp_search = GridSearchCV(
    exp_pipeline,
    {'features__kw_args': [{'k': k} for k in k_grid]},
    scoring='neg_mean_squared_error',
    cv=cv,
)
exp_search.fit(X_train, y_train)

best_k = exp_search.best_params_['features__kw_args']['k']
exp_model = exp_search.best_estimator_
print(f'Лучший k = {best_k}, MSE на кросс-валидации = {-exp_search.best_s
```

```
Лучший k = 0.525, MSE на кросс-валидации = 1.3059
```

In [12]:
```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(k_grid, -exp_search.cv_results_['mean_test_score'])
ax.axvline(best_k, color='red', linestyle='--', label=f'k = {best_k}')
ax.set_xlabel('k')
ax.set_ylabel('MSE на кросс-валидации')
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



6. Проверка качества на тестовой выборке

In [13]:
```
models = {
    'Линейная': linear,
    f'Полином степени {best_degree}': poly_model,
    f'a·exp(−{best_k}x) + bx + c': exp_model,
}

metrics = pd.DataFrame([row for name, model in models.items() for row in
metrics.pivot(index='model', columns='split', values=['MSE', 'MAE', 'R2']
```

Out[13]:

	MSE		MAE		R2	
split	test	train	test	train	test	train
model						
a·exp(-0.525x) + bx + c	1.8227	1.2693	1.0154	0.9067	0.9735	0.9794
Линейная	14.3183	13.9074	2.8959	3.0560	0.7915	0.7744
Полином степени 4	1.6420	1.0551	0.9228	0.8244	0.9761	0.9829

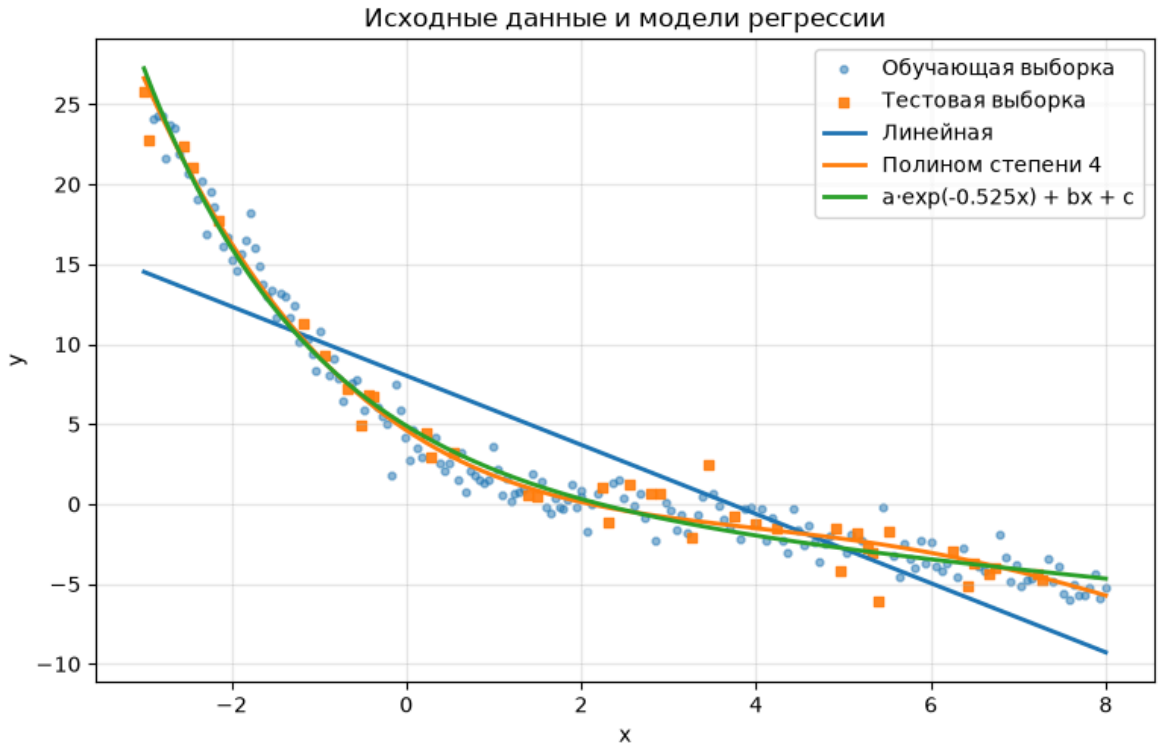
Простая прямая заметно недообучена: у неё большая ошибка и на обучающей, и на тестовой выборке. Обе нелинейные модели снижают MSE примерно в 8 раз и объясняют около 97–98 % дисперсии y ($R^2 \approx 0.97$–0.98). Ошибки на обучающей и тестовой выборках близки, значит переобучения нет.

7. Визуализация результата

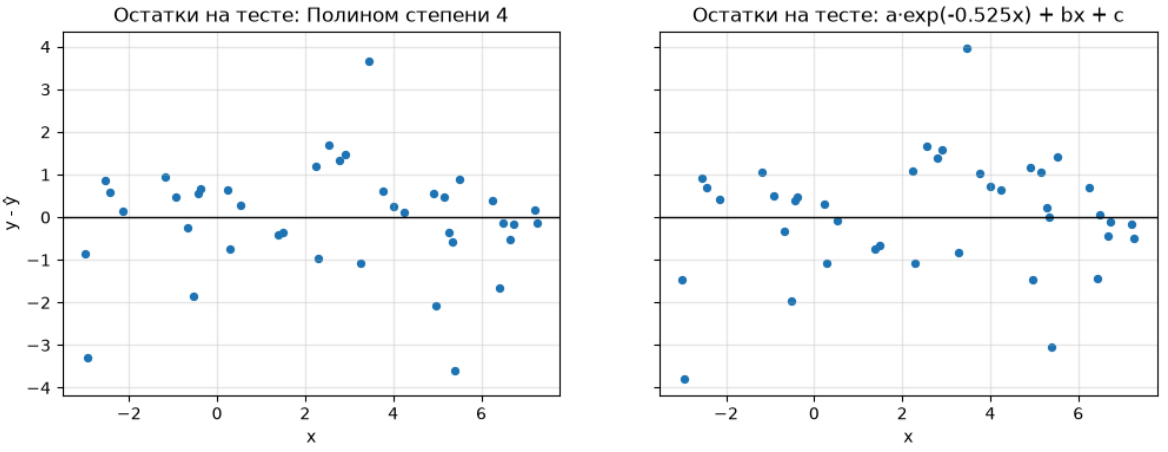
In [14]:
```
x_grid = pd.DataFrame({'x': np.linspace(data['x'].min(), data['x'].max(),

fig, ax = plt.subplots(figsize=(9, 5.5))
ax.scatter(X_train['x'], y_train, s=12, alpha=0.5, label='Обучающая выбор
ax.scatter(X_test['x'], y_test, s=18, alpha=0.9, marker='s', label='Тесто
for name, model in models.items():
```

```
    ax.plot(x_grid['x'], model.predict(x_grid), linewidth=2, label=name)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Исходные данные и модели регрессии')
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



In [15]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
for ax, (name, model) in zip(axes, list(models.items())[1:]):
    residuals = y_test - model.predict(X_test)
    ax.scatter(X_test['x'], residuals, s=18)
    ax.axhline(0, color='black', linewidth=1)
    ax.set_title(f'Остатки на тесте: {name}')
    ax.set_xlabel('x')
    ax.grid(alpha=0.3)
axes[0].set_ylabel('y - ŷ')
plt.show()
```



Остатки обеих нелинейных моделей разбросаны вокруг нуля без явной закономерности, то есть основную форму зависимости модели уловили.

Эмпирическая формула

In [16]:
```
poly_coefs = poly_model.named_steps['linreg'].coef_
poly_intercept = poly_model.named_steps['linreg'].intercept_
poly_terms = ' '.join(f'{c:+.4f}·x^{i}' for i, c in enumerate(poly_coefs,
print(f'Полином:    y = {poly_intercept:.4f} {poly_terms}')

a, b = exp_model.named_steps['linreg'].coef_
c = exp_model.named_steps['linreg'].intercept_
print(f'Экспонента: y = {a:.4f}·exp(-{best_k}·x) {b:+.4f}·x {c:+.4f}')
```

```
Полином:    y = 4.6196 -3.6209·x^1 +0.8840·x^2 -0.1066·x^3 +0.0041·x^4
Экспонента: y = 5.4225·exp(-0.525·x) -0.5276·x -0.5219
```

Вывод

1. Данные варианта 8 описывают убывающую нелинейную зависимость. Обычная линейная регрессия для неё не подходит ($R^2 \approx 0.79$ на тесте).
2. Модели, линейные по параметрам, но нелинейные по x, хорошо описывают данные. Полином 4-й степени немного точнее (MSE на тесте ≈ 1.64, $R^2 \approx 0.976$), модель $y = ae^{-kx} + bx + c$ отстаёт от него совсем немного (MSE ≈ 1.82, $R^2 \approx 0.974$).
3. В качестве итоговой эмпирической формулы выбрана

   $$y \approx 5.42\, e^{-0.525x} - 0.528\, x - 0.522.$$

   Она чуть уступает полиному по ошибке, но параметров в ней четыре, а не пять, и у каждого есть понятный смысл: экспоненциально затухающая составляющая и линейный тренд. Такая модель правдоподобнее ведёт себя за пределами отрезка $[-3, 8]$, тогда как полином 4-й степени при $|x| \to \infty$ уходит в $+\infty$.